Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №3
по дисциплине
«Современные системы программирования»

Выполнил: ст. гр. 320601
П. Д. Дудников
Р. В. Земляник

Проверила: Ю. О. Герман

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ 3
2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 4
3 ВЫПОЛНЕНИЕ РАБОТЫ 5
3.1 Структура программы 5
3.2 Примеры методических указаний 5
3.3 Задание 1. Класс с приватными полями и свойствами 9
3.4 Задание 2. Информация о методах класса 10
3.5 Задание 3. Создание экземпляра через *Activator.CreateInstance* 11
3.6 Задания 4 и 5. Доступ к приватному полю 12
3.7 Задание 6. Вызов приватного метода 13
3.8 Задание 7. Динамическая загрузка плагинов 13
3.9 Дополнительное задание. *PluginManager* 14
4 КОНТРОЛЬНЫЕ ВОПРОСЫ 16
ЗАКЛЮЧЕНИЕ 18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 19
ПРИЛОЖЕНИЕ А 20

1 ЦЕЛЬ РАБОТЫ

Цель лабораторной работы – изучить механизм рефлексии в *C#*: получение метаданных о типах во время выполнения, инспекцию членов классов (полей, методов, свойств, конструкторов), позднее связывание для динамического создания объектов и вызова методов, а также работу с приватными членами через рефлексию.

Для достижения цели в работе решены следующие задачи:

– выполнить примеры методических указаний по инспекции типа, атрибутам, приватным членам и позднему связыванию;

– создать класс с приватными полями и свойствами с валидацией и вывести информацию о его методах;

– создать экземпляры класса через *Activator.CreateInstance* без оператора *new*;

– получить доступ к приватному полю и приватным методам класса-«черного ящика»;

– реализовать систему динамической загрузки плагинов по именам из конфигурационного файла;

– собрать из частей и выполнить дополнительный пример *PluginManager*.

Все задания выполнены в одном консольном приложении *Lab3* для платформы *.NET* 10, полный текст программы приведен в приложении А.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Рефлексия – механизм, позволяющий получать информацию о типах данных во время выполнения программы и динамически работать с ними [1]. Сборка *.NET* содержит метаданные обо всех своих типах и их членах, а классы пространства имен *System.Reflection* предоставляют к ним доступ [2].

Центральный класс рефлексии – *System.Type*. Объект *Type* можно получить тремя способами: оператором *typeof(T)* на этапе компиляции, методом *GetType()* у существующего экземпляра и статическим методом *Type.GetType(*"Имя.Типа") по строковому имени во время выполнения [1]. Свойства *Name, FullName, Namespace, BaseType, IsClass, IsAbstract* и *IsSealed* описывают сам тип, а методы *GetFields(), GetMethods(), GetProperties(), GetConstructors(), GetEvents()* и *GetInterfaces()* возвращают его члены в виде объектов *FieldInfo, MethodInfo, PropertyInfo* и *ConstructorInfo*.

Набор возвращаемых членов задается флагами *BindingFlags*. По умолчанию методы *Get...()* возвращают только открытые члены, для получения закрытых указывают *BindingFlags.NonPublic* вместе с *Instance* или *Static*, а флаг *DeclaredOnly* исключает унаследованные члены [1, 2].

Позднее связывание (*late binding*) – механизм, при котором связывание вызова метода с его реализацией происходит во время выполнения программы, а не во время компиляции [1]. Тип находится по имени, экземпляр создается методом *Activator.CreateInstance()* или *ConstructorInfo.Invoke()*, а методы вызываются через *MethodInfo.Invoke()*. При раннем связывании компилятор проверяет имена и типы, при позднем ошибки обнаруживаются только при выполнении.

Атрибуты – классы, производные от *System.Attribute*, которые добавляют к типам и членам метаданные. Их читают методом *GetCustomAttribute<T>()*, что позволяет, например, отмечать классы плагинов и отбирать их при загрузке сборки [1].

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Структура программы

Программа состоит из одного файла *Program.cs*. Примеры методических указаний находятся в пространстве имен *LabReflection* (классы *Product* и *DynamicCalculator*), в *LabReflection.AttributeExample* (атрибуты и плагин *MyPlugin*) и в *LabReflection.PluginSystem* (дополнительное задание). Собственные классы заданий 1–7 размещены в *LabReflection.Tasks*. Пространство имен *LabReflection* сохранено, так как пример позднего связывания ищет тип по строке "*LabReflection.DynamicCalculator*".

Методы *Main* примеров переименованы в *Run*, точка входа *LabReflection.Program.Main* выводит меню, по номеру пункта которого запускается пример или задание. Для задания 7 рядом с программой копируется файл *plugins.config*.

3.2 Примеры методических указаний

Первый пример выполняет базовую инспекцию класса *Product*. Результат приведен на рисунках 1 и 2.

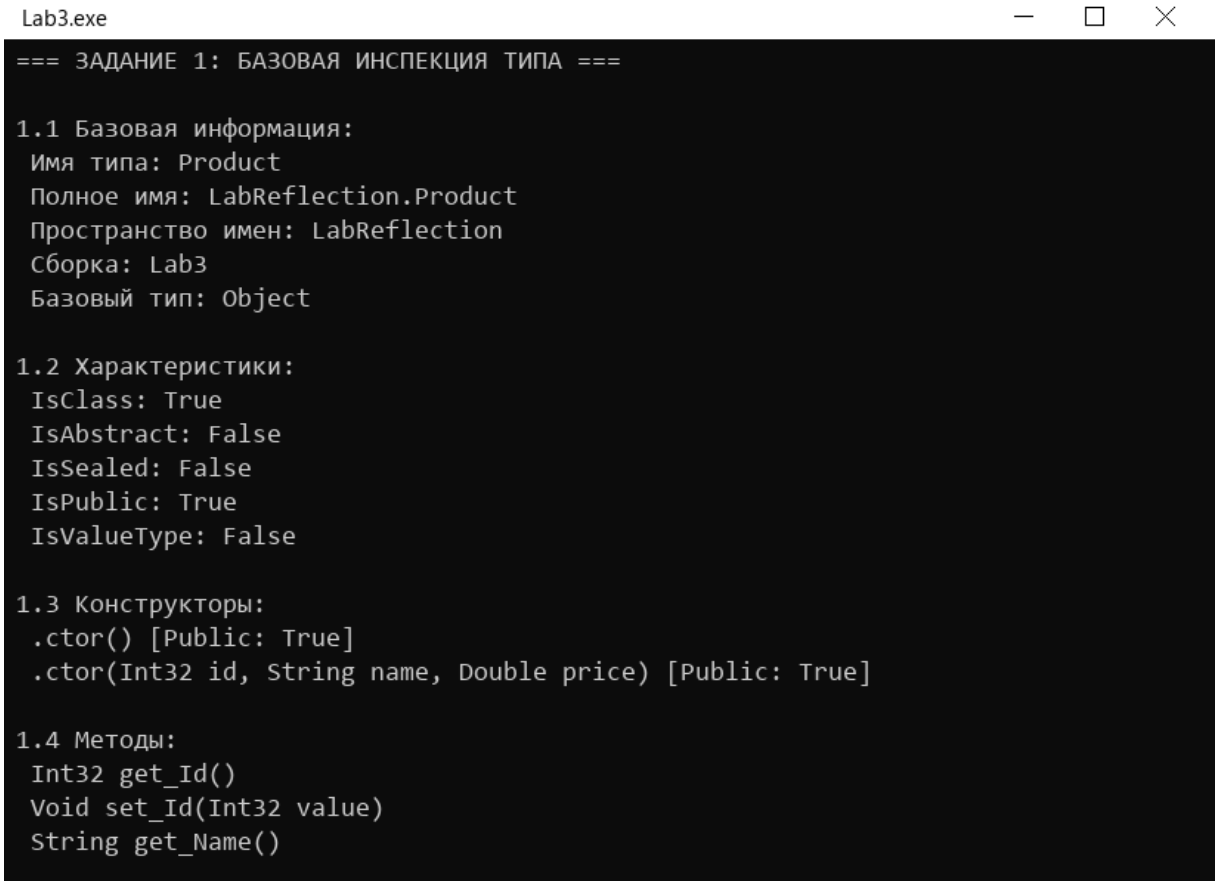


Рисунок 1 – Базовая инспекция типа *Product* (начало)

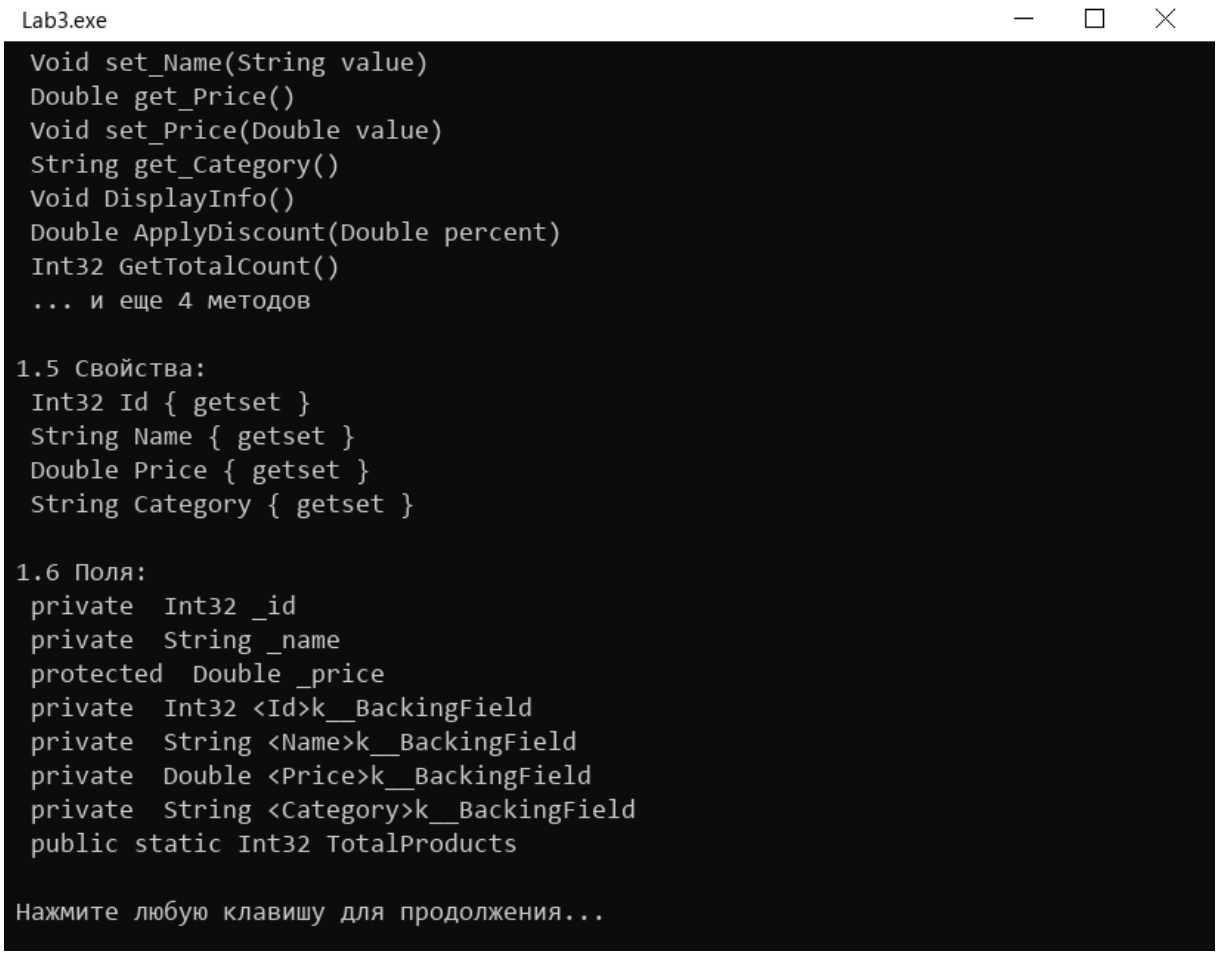


Рисунок 2 – Базовая инспекция типа *Product* (окончание)

Тип *Product* находится в пространстве имен *LabReflection* сборки *Lab3*, наследуется от *Object*, является открытым неабстрактным классом. Метод *GetMethods()* без флагов вернул 14 открытых методов, среди которых методы доступа свойств *get_Id* и *set_Id* и методы, унаследованные от *Object*. Среди полей кроме объявленных *_id, _name, _price* и *TotalProducts* видны поля *<Id>k__BackingField*, которые компилятор создает для автосвойств. Свойство *Category* отмечено как *getset*, так как его закрытый метод *set* тоже существует.

Пример с интерфейсом и атрибутами читает атрибут [*Plugin]* класса *MyPlugin* и атрибуты [*PluginMethod]* его методов, а затем вызывает методы через *Invoke*. Результат приведен на рисунке 3.

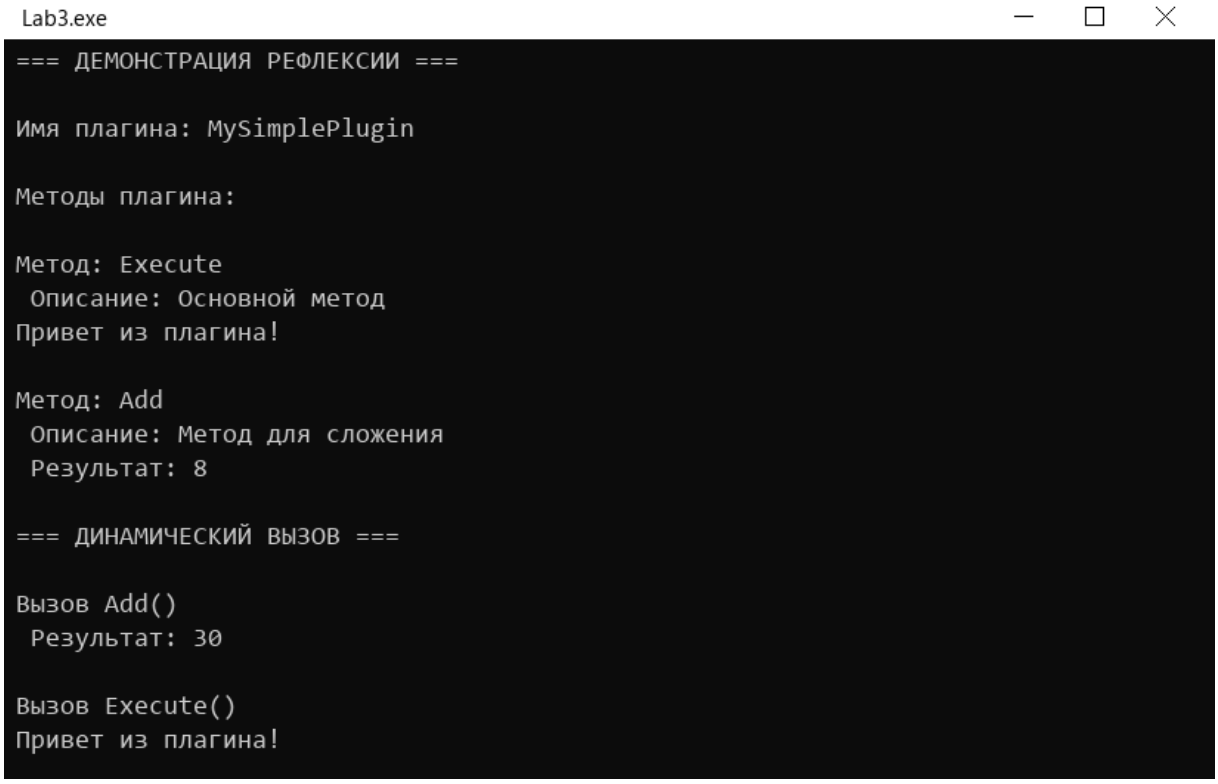


Рисунок 3 – Чтение атрибутов и вызов методов плагина

Из метаданных получены имя плагина *MySimplePlugin* и описания методов. Метод *Add(5*, 3) вернул 8, а универсальный метод *CallMethodByName* вызвал *Add(10*, 20) и *Execute()* только по строковому имени.

Пример работы с приватными членами изменяет закрытые поля *_price* и *_name* и вызывает закрытый метод *ValidatePrice* и защищенный метод *UpdateCategory*, как показано на рисунке 4.

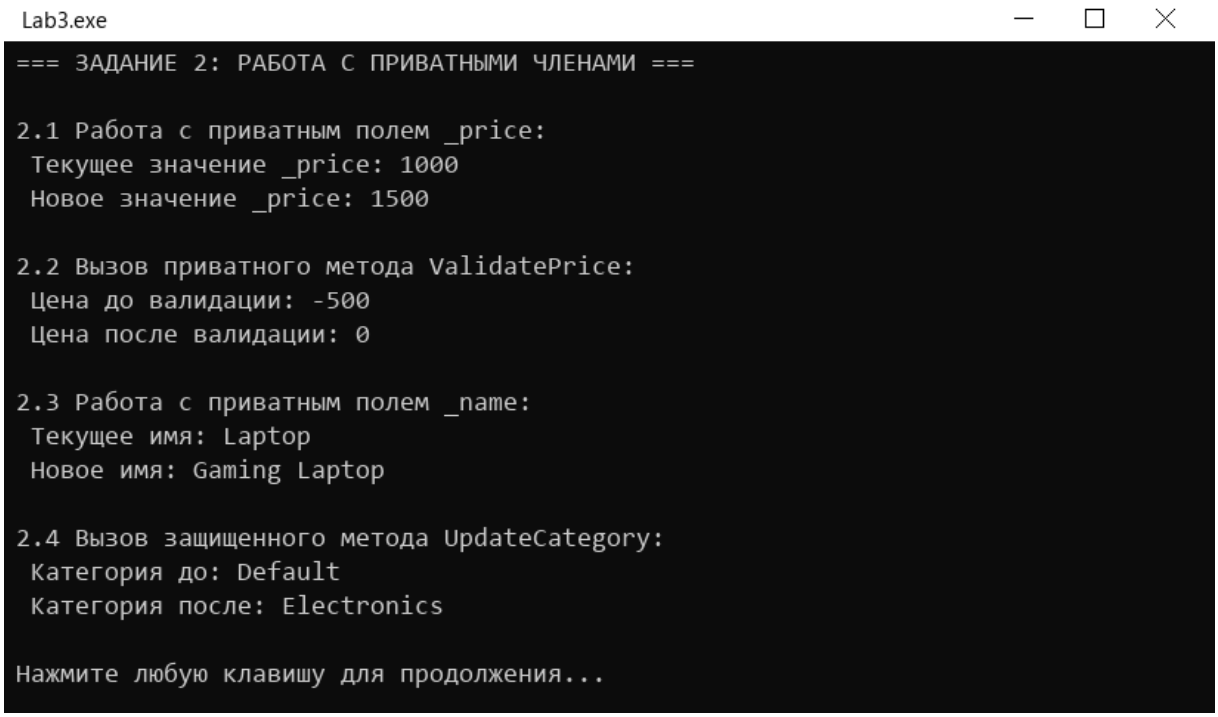


Рисунок 4 – Работа с приватными членами класса *Product*

Цена 1000 изменена на 1500, затем на -500, после вызова *ValidatePrice* стала равна 0. Имя изменено на *Gaming Laptop*, категория – с *Default* на *Electronics*, хотя у свойства *Category* закрытый метод *set*. Это подтверждает, что рефлексия обходит модификаторы доступа.

Пример позднего связывания находит тип *DynamicCalculator* по строке, создает его экземпляры и вызывает методы. Результат приведен на рисунке 5.

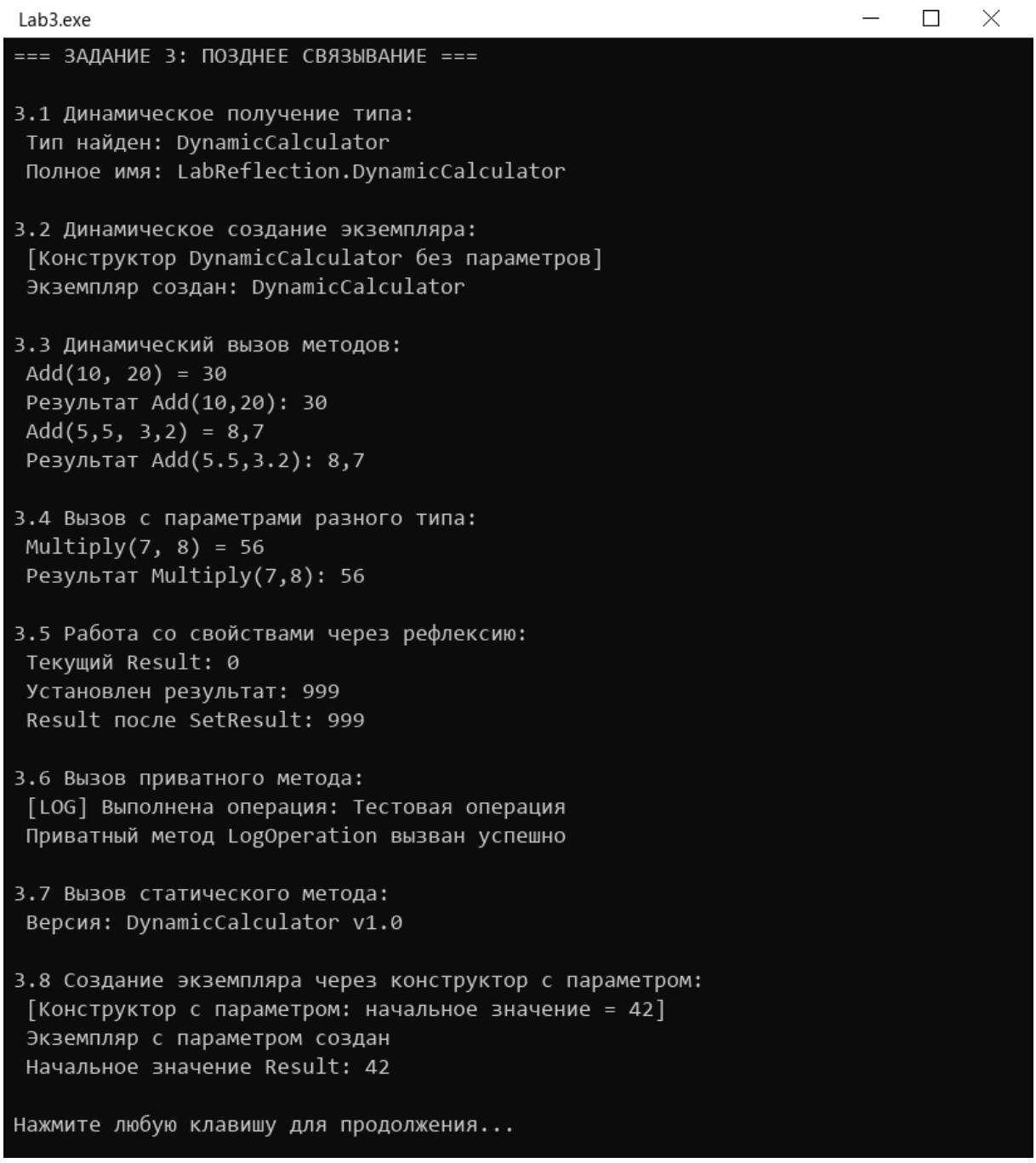


Рисунок 5 – Позднее связывание с классом *DynamicCalculator*

Перегрузки *Add(int, int)* и *Add(double, double)* выбраны по массиву типов параметров и вернули 30 и 8,7. Через рефлексию прочитано свойство *Result*, вызваны *SetResult(999)*, закрытый *LogOperation* и статический *GetVersion*, а конструктор с параметром 42 вызван через *ConstructorInfo.Invoke*.

3.3 Задание 1. Класс с приватными полями и свойствами

Создан класс *Employee* с тремя закрытыми полями разных типов: *string _name, int _age* и *double _salary*. Для каждого поля реализовано открытое свойство с валидацией: имя не может быть пустым, возраст должен быть от

18 до 70, зарплата не может быть отрицательной. Класс также содержит открытые, закрытый, защищенный и статический методы для последующих заданий. Результат проверки показан на рисунке 6.

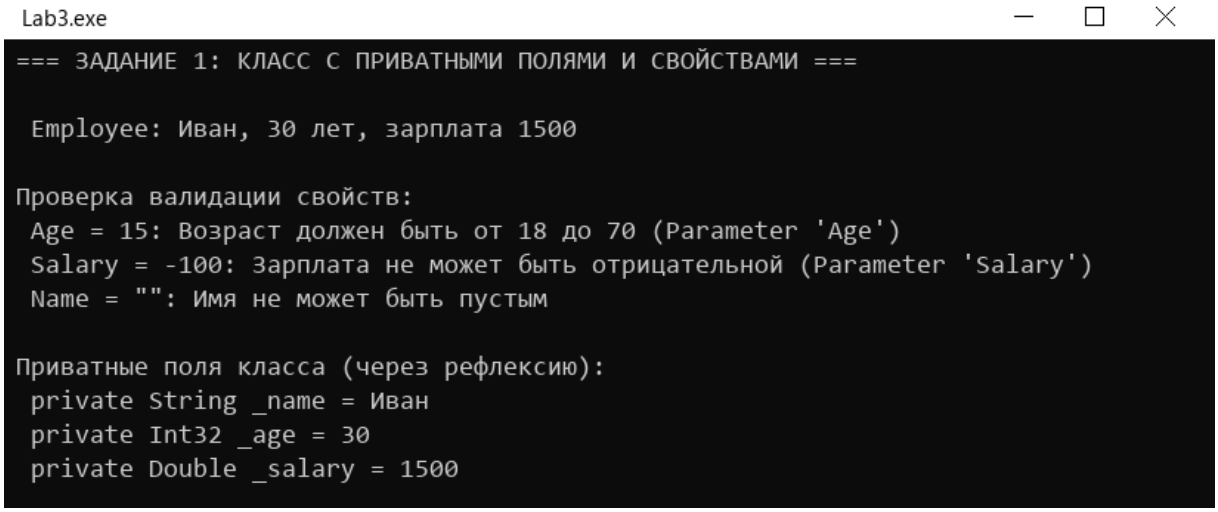


Рисунок 6 – Класс *Employee* и проверка валидации свойств

Некорректные значения *Age* = 15, *Salary* = -100 и *Name* = "" отклонены исключениями, а значения закрытых полей прочитаны через рефлексию без открытых свойств.

3.4 Задание 2. Информация о методах класса

Метод *GetMethods()* вызван с флагами *Public, NonPublic, Instance, Static* и *DeclaredOnly*. Для каждого метода выводятся модификатор доступа, признак *static*, возвращаемый тип, имя и параметры, а методы *get_* и *set_* отфильтрованы по свойству *IsSpecialName*. Результат приведен на рисунке 7 и в таблице 1.

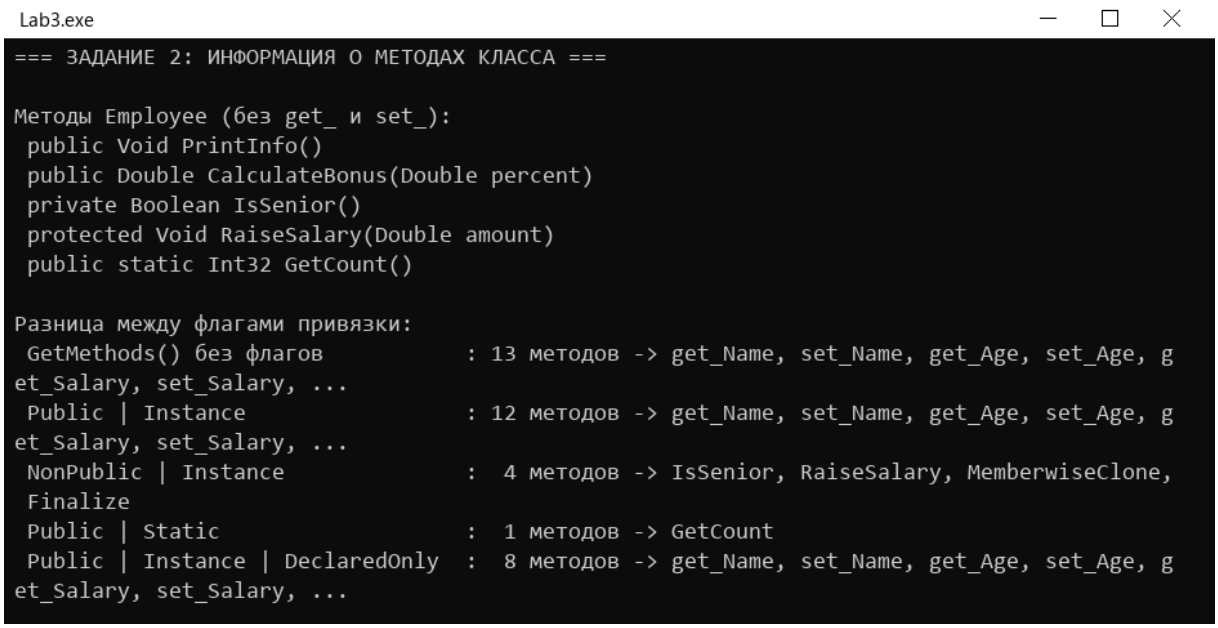


Рисунок 7 – Методы класса *Employee*

Таблица 1 – Методы класса *Employee* при разных флагах привязки

Флаги *BindingFlags*	Число методов	Что попадает в результат
без флагов	13	открытые экземплярные и статические, включая унаследованные от *Object*
Public | *Instance*	12	открытые экземплярные, включая *get_* и *set_* и методы *Object*
NonPublic | *Instance*	4	*IsSenior, RaiseSalary, MemberwiseClone, Finalize*
Public | *Static*	1	*GetCount*
Public | *Instance* | *DeclaredOnly*	8	только объявленные в *Employee*, без методов *Object*

Без флага *NonPublic* закрытый *IsSenior* и защищенный *RaiseSalary* не видны, а без *DeclaredOnly* в результат попадают *ToString, Equals, GetHashCode* и *GetType* базового класса.

3.5 Задание 3. Создание экземпляра через *Activator.CreateInstance*

Объекты *Employee* созданы без оператора *new* двумя способами: *Activator.CreateInstance(type)* вызывает конструктор без параметров, а *Activator.CreateInstance(type, args)* подбирает конструктор по аргументам. После создания значения свойств установлены методом *PropertyInfo.SetValue* и прочитаны через *GetValue*, как показано на рисунке 8.

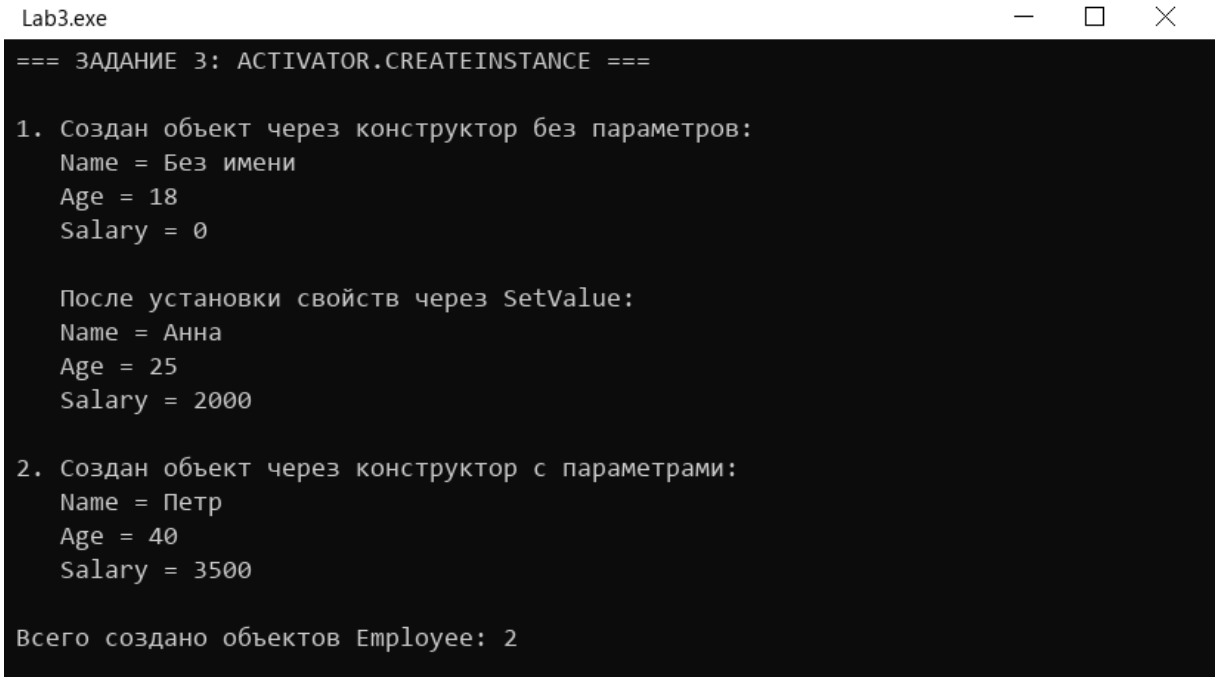


Рисунок 8 – Создание объектов через *Activator.CreateInstance*

Первый объект создан со значениями по умолчанию и после *SetValue* получил имя Анна, возраст 25 и зарплату 2000. Второй объект сразу получил значения конструктора. Статический метод *GetCount*, вызванный через *Invoke(null, null)*, подтвердил создание двух объектов.

3.6 Задания 4 и 5. Доступ к приватному полю

Для задания 4 создан класс *SecretBox* – «черный ящик» с закрытым полем *_secret* = 42, открытым методом *GetSecret()* для проверки и закрытыми методами *Calculate(a, b)*, *Square()* и *Describe(prefix)*. В задании 5 поле получено через *GetField* с флагами *NonPublic* и *Instance* и изменено методом *SetValue*. Результат приведен на рисунке 9.

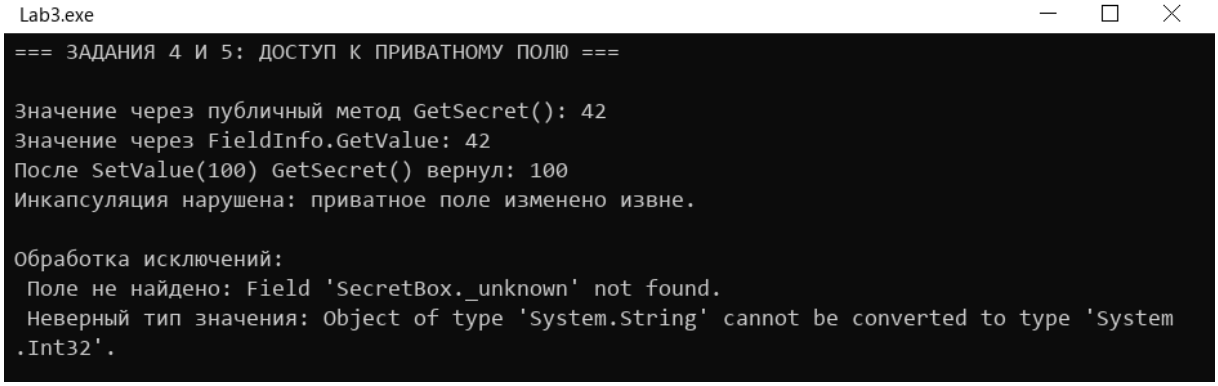


Рисунок 9 – Изменение приватного поля через *FieldInfo*

После *SetValue(100)* открытый метод *GetSecret()* вернул 100, то есть закрытое поле изменено извне и инкапсуляция нарушена. Обработаны исключения: обращение к несуществующему полю *_unknown* (*MissingFieldException*) и запись строки в поле типа *int* (*ArgumentException*).

Обработчик *FieldAccessException* предусмотрен для случая недостаточных прав.

3.7 Задание 6. Вызов приватного метода

Закрытые методы *SecretBox* с нулем, одним и двумя параметрами вызваны двумя способами: через *MethodInfo.Invoke* и через делегат, созданный методом *CreateDelegate*. Результат показан на рисунке 10.

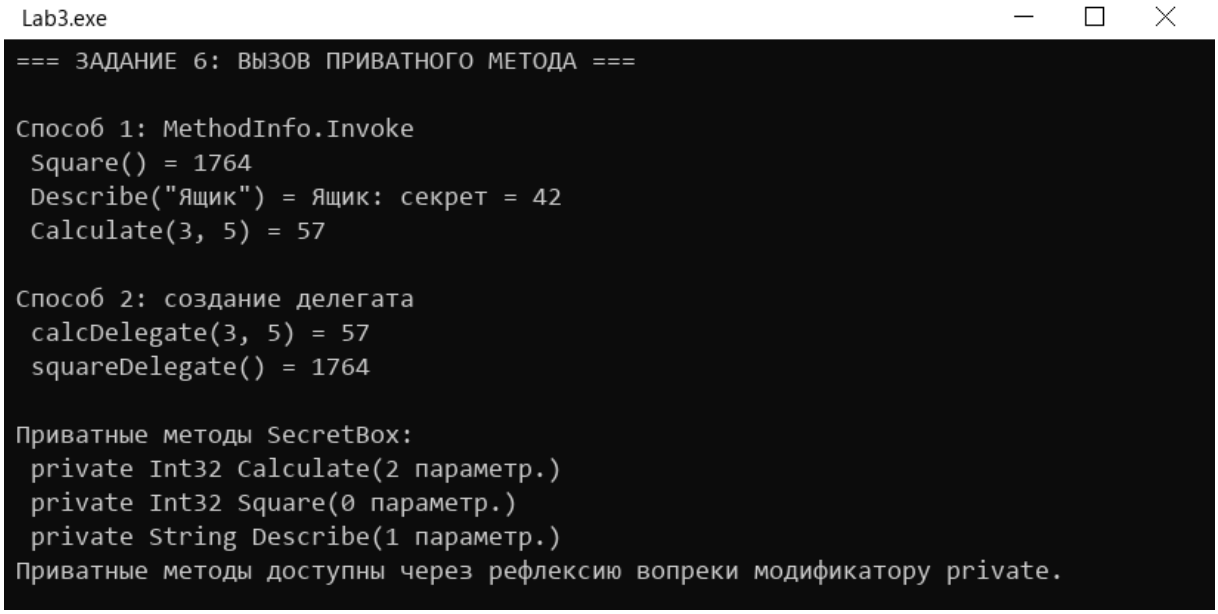


Рисунок 10 – Вызов приватных методов через *Invoke* и делегат

Square() вернул 1764, *Describe("*Ящик") – строку с секретом, *Calculate(3*, 5) – 57. Делегаты *Func<int, int, int>* и *Func<int>* дали те же результаты. Делегат связывается с методом один раз, поэтому повторные вызовы через него выполняются быстрее, чем через *Invoke*, где каждый раз упаковываются аргументы и проверяются типы.

3.8 Задание 7. Динамическая загрузка плагинов

Система плагинов использует интерфейс *IPlugin* и атрибут [*Plugin]* из примера методических указаний. Метод *Assembly.GetTypes()* возвращает все типы сборки, из них отбираются неабстрактные классы, реализующие *IPlugin* и отмеченные атрибутом, который проверяется через *GetCustomAttribute*. Затем из файла *plugins.config* читаются имена классов, и для каждого найденного типа плагин создается методом *Activator.CreateInstance*. Содержимое конфигурационного файла приведено в приложении А, результат – на рисунке 11.

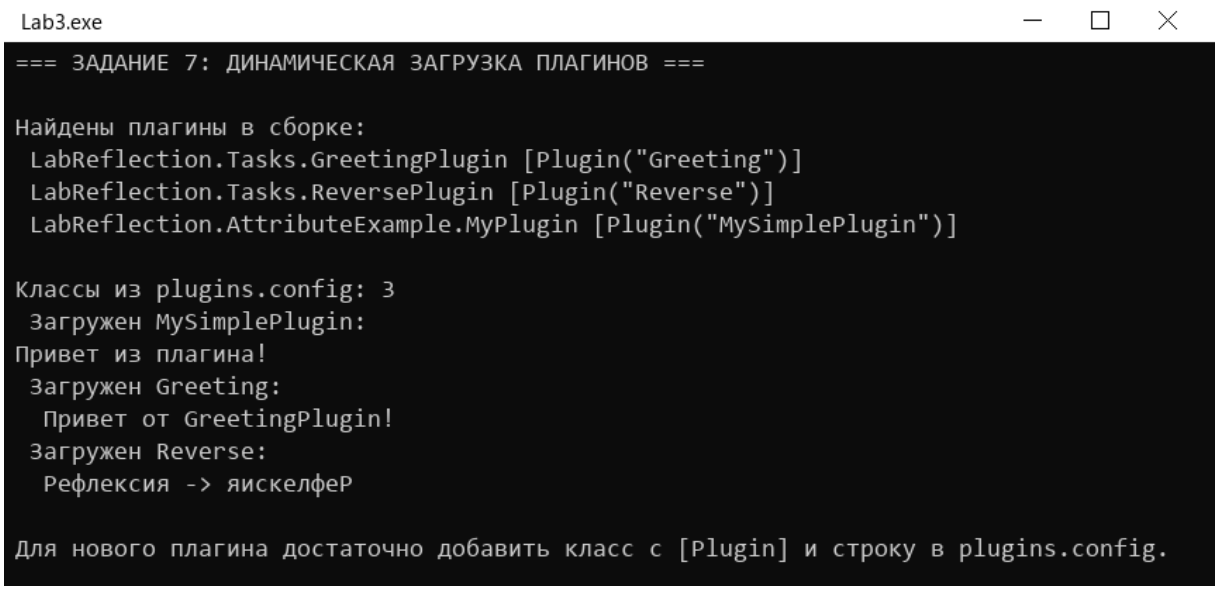


Рисунок 11 – Загрузка плагинов из конфигурационного файла

В сборке найдены три плагина с атрибутом, класс *NoAttributePlugin* отфильтрован, так как не имеет атрибута [*Plugin]*. Все три класса из *plugins.config* загружены и выполнены. Для добавления нового плагина достаточно создать класс с атрибутом и добавить его имя в файл, код загрузчика при этом не изменяется.

3.9 Дополнительное задание. *PluginManager*

Из частей методических указаний собран полный код: интерфейсы *IPlugin* и *IParameterizedPlugin*, плагины *CalculatorPlugin* и *LoggerPlugin* и класс *PluginManager*. Чтобы получить показанный в методических указаниях вывод «*Loading built-in plugins...*», в *PluginManager* добавлен метод *LoadBuiltInPlugins()*, который применяет ту же проверку типов к текущей сборке. Результат приведен на рисунке 12.

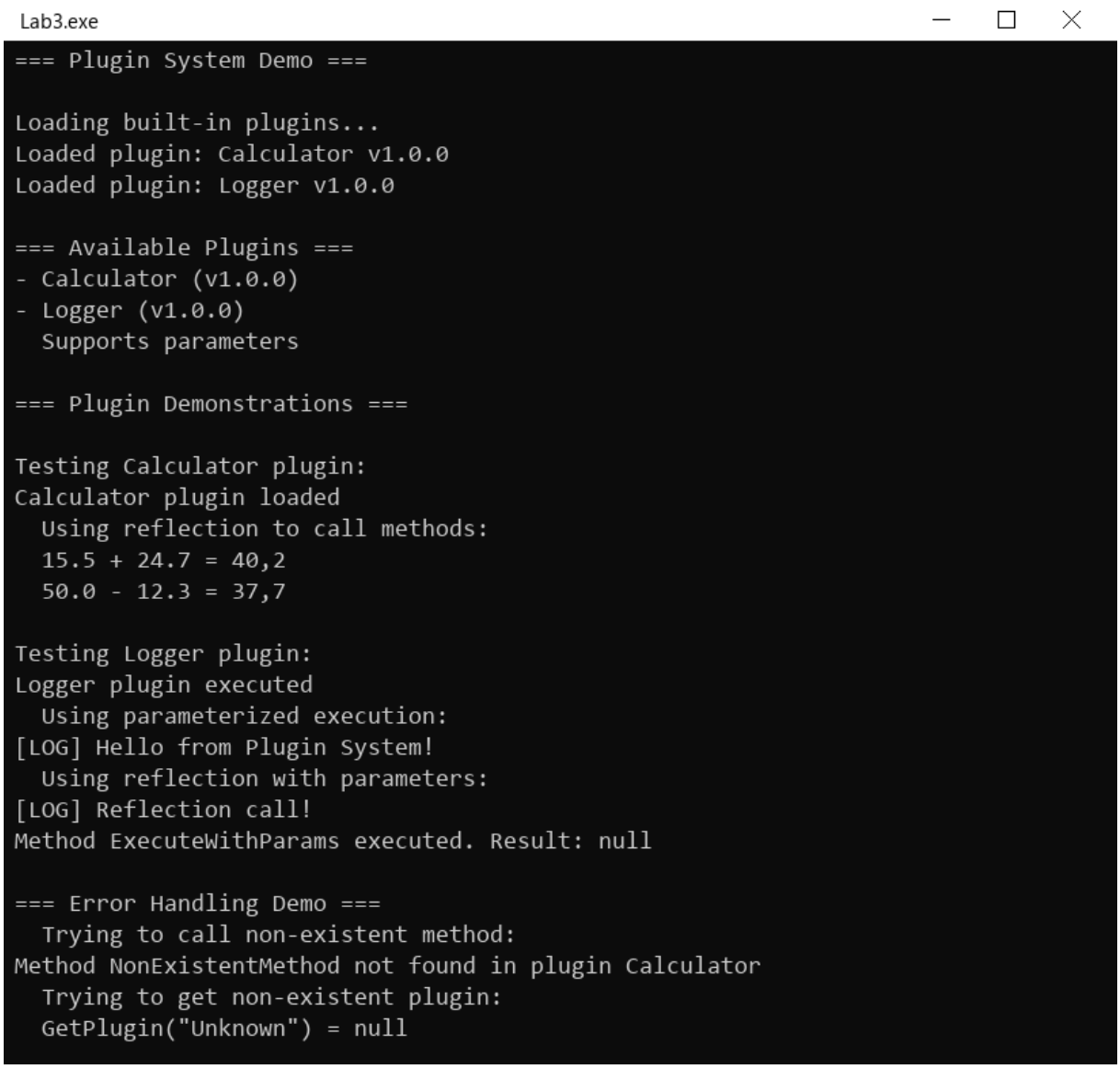


Рисунок 12 – Работа *PluginManager*

Загружены плагины *Calculator* и *Logger*, для *Logger* определена поддержка параметров. Методы *Add* и *Subtract* вызваны через обобщенный *ExecutePluginMethod<double>* и вернули 40,2 и 37,7, метод *ExecuteWithParams* вызван через рефлексию со словарем параметров. Вызов несуществующего метода и запрос неизвестного плагина обработаны без исключений.

4 КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1. Что такое рефлексия и для чего она используется в *C#*?

Рефлексия – механизм получения информации о типах во время выполнения и динамической работы с ними. Она используется в системах плагинов, сериализаторах, *ORM*, контейнерах внедрения зависимостей, тестовых фреймворках и инструментах анализа сборок, то есть там, где типы заранее неизвестны программе.

Вопрос 2. Какие пространства имен и классы необходимы для работы с рефлексией?

Основное пространство имен – *System.Reflection* с классами *Assembly*, *MemberInfo, FieldInfo, MethodInfo, PropertyInfo, ConstructorInfo, ParameterInfo* и перечислением *BindingFlags*. Класс *Type* и класс *Activator* находятся в пространстве имен *System*.

Вопрос 3. В чем разница между *GetType()* и *typeof()*?

Оператор *typeof(T)* получает *Type* по имени типа на этапе компиляции и не требует объекта. Метод *GetType()* вызывается у экземпляра и возвращает фактический тип объекта во время выполнения, который может быть наследником типа переменной.

Вопрос 4. Как получить доступ к приватным членам класса через рефлексию?

Нужно вызвать *GetField, GetMethod* или *GetProperty* с флагами *BindingFlags.NonPublic* и *BindingFlags.Instance* (или *Static*), а затем использовать *GetValue* и *SetValue* для полей и *Invoke* для методов, как в заданиях 5 и 6.

Вопрос 5. Какие ограничения и риски связаны с использованием рефлексии?

Рефлексия нарушает инкапсуляцию и позволяет перевести объект в недопустимое состояние в обход валидации. Ошибки в именах и типах обнаруживаются только при выполнении, код становится хрупким при переименовании членов, а при усечении сборок (*trimming*) и *AOT*-компиляции нужные метаданные могут быть удалены.

Вопрос 6. Что такое *BindingFlags* и для чего они используются?

BindingFlags – перечисление флагов, задающих, какие члены искать: открытые (*Public*) или закрытые (*NonPublic*), экземплярные (*Instance*) или статические (*Static*), только объявленные в типе (*DeclaredOnly*), с учетом регистра или без (*IgnoreCase*). Разница результатов при разных флагах показана в таблице 1.

Вопрос 7. Как динамически создать экземпляр класса через рефлексию?

Получить *Type* (например, *Type.GetType* по имени) и вызвать *Activator.CreateInstance(type)* или *Activator.CreateInstance(type, args)*. Другой способ – получить *ConstructorInfo* методом *GetConstructor* с массивом типов параметров и вызвать его метод *Invoke*.

Вопрос 8. В чем отличие позднего связывания от раннего?

При раннем связывании тип и метод известны компилятору, он проверяет вызов и формирует прямой вызов. При позднем связывании тип и метод определяются по именам во время выполнения через рефлексию, проверка выполняется только при вызове, а сам вызов медленнее.

Вопрос 9. Какие недостатки есть у рефлексии?

Основные недостатки – низкая скорость вызова по сравнению с прямым, отсутствие проверки на этапе компиляции, упаковка аргументов в *object[]*, нарушение инкапсуляции и сложность сопровождения кода. Для частых вызовов используют делегаты, созданные через *CreateDelegate*, или кэширование *MethodInfo*.

ЗАКЛЮЧЕНИЕ

В лабораторной работе изучен механизм рефлексии *C#*. Выполнены примеры методических указаний: инспекция типа *Product*, чтение атрибутов, работа с закрытыми членами и позднее связывание с классом *DynamicCalculator*.

В собственных заданиях создан класс *Employee* с валидацией свойств, получен полный список его методов и показано влияние флагов *BindingFlags* на результат. Объекты созданы без оператора *new* через *Activator.CreateInstance*, закрытое поле класса *SecretBox* изменено извне, а закрытые методы вызваны через *Invoke* и через делегат. Система плагинов загружает классы по именам из *plugins.config* и фильтрует их по интерфейсу и атрибуту, что позволяет добавлять плагины без изменения загрузчика.

Результаты показали, что рефлексия дает полный доступ к метаданным и закрытым членам, но ошибки обнаруживаются только при выполнении и инкапсуляция перестает защищать состояние объекта, поэтому ее применяют там, где без динамики не обойтись.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Рефлексия в *C#*. Механизм рефлексии (*System.Type*) и позднее связывание : методические указания к лабораторной работе № 3 по дисциплине «Современные системы программирования». – Минск : БГУИР, 2026.

2 Отражение в *.NET* [Электронный ресурс] // *Microsoft Learn*. – Режим доступа: *https://learn.microsoft.com/ru-ru/dotnet/fundamentals/reflection/reflection*. – Дата доступа: 22.10.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программы

Файл *Program.cs*:

```
// Лабораторная работа 3. Рефлексия в C#. Механизм рефлексии (System.Type) и
позднее связывание
// Все задания собраны в одном файле. Примеры методички лежат в своих
пространствах имен,
// их методы Main переименованы в Run и вызываются из общего меню.

using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Text;

namespace LabReflection
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = Encoding.UTF8;

            while (true)
            {
                Console.WriteLine("\n=== Лабораторная работа 3: Рефлексия и
позднее связывание ===");
                Console.WriteLine("1 - Пример: базовая инспекция типа
Product");
                Console.WriteLine("2 - Пример: интерфейс, атрибуты и
Invoke");
                Console.WriteLine("3 - Пример: работа с приватными членами
Product");
                Console.WriteLine("4 - Пример: позднее связывание
(DynamicCalculator)");
                Console.WriteLine("5 - Задание 1. Класс с приватными полями и
свойствами");
                Console.WriteLine("6 - Задание 2. Информация о методах
класса");
                Console.WriteLine("7 - Задание 3. Activator.CreateInstance");
                Console.WriteLine("8 - Задания 4 и 5. Доступ к приватному
полю");
                Console.WriteLine("9 - Задание 6. Вызов приватного метода");
                Console.WriteLine("10 - Задание 7. Загрузка плагинов из
конфигурации");
                Console.WriteLine("11 - Дополнительно: PluginManager");
                Console.WriteLine("0 - Выход");
                Console.Write("> ");

                string choice = Console.ReadLine();
                if (choice == null || choice == "0") break;

                switch (choice)
                {
                    case "1": Task1Inspection.Run(); break;
                    case "2": AttributeExample.Program.Run(); break;
                    case "3": Task2.Run(); break;
```

```
                    case "4": Task3.Run(); break;
                    case "5": Tasks.Task1.Run(); break;
                    case "6": Tasks.Task2.Run(); break;
                    case "7": Tasks.Task3.Run(); break;
                    case "8": Tasks.Task5.Run(); break;
                    case "9": Tasks.Task6.Run(); break;
                    case "10": Tasks.Task7.Run(); break;
                    case "11": PluginSystem.Program.Run(); break;
                    default: Console.WriteLine("Неизвестный пункт меню");
break;
                }
            }
        }
    }

    // ==================== Пример инспекции типа (методичка)
====================

    // Класс для исследования
    public class Product
    {
        // Поля
        private int _id;
        private string _name;
        protected double _price;
        public static int TotalProducts = 0;

        // Свойства
        public int Id { get; set; }
        public string Name { get; set; }
        public double Price { get; set; }
        public string Category { get; private set; }

        // Конструкторы
        public Product()
        {
            _id = 0;
            _name = "Unknown";
            Category = "Default";
            TotalProducts++;
        }

        public Product(int id, string name, double price)
        {
            _id = id;
            _name = name;
            _price = price;
            Id = id;
            Name = name;
            Price = price;
            Category = "Default";
            TotalProducts++;
        }

        // Методы
        public void DisplayInfo()
        {
            Console.WriteLine($"Product: {_name}, Price: {_price}");
        }

        public double ApplyDiscount(double percent)
        {
```

```
            return _price * (1 - percent / 100);
        }

        private void ValidatePrice()
        {
            if (_price < 0) _price = 0;
        }

        protected void UpdateCategory(string newCategory)
        {
            Category = newCategory;
        }

        public static int GetTotalCount()
        {
            return TotalProducts;
        }
    }

    class Task1Inspection
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 1: БАЗОВАЯ ИНСПЕКЦИЯ ТИПА ===\n");

            Type productType = typeof(Product);

            // 1.1 Базовая информация о типе
            Console.WriteLine("1.1 Базовая информация:");
            Console.WriteLine($" Имя типа: {productType.Name}");
            Console.WriteLine($" Полное имя: {productType.FullName}");
            Console.WriteLine($" Пространство имен:
{productType.Namespace}");
            Console.WriteLine($" Сборка:
{productType.Assembly.GetName().Name}");
            Console.WriteLine($" Базовый тип: {productType.BaseType?.Name ??
"None"}");

            // 1.2 Характеристики типа
            Console.WriteLine("\n1.2 Характеристики:");
            Console.WriteLine($" IsClass: {productType.IsClass}");
            Console.WriteLine($" IsAbstract: {productType.IsAbstract}");
            Console.WriteLine($" IsSealed: {productType.IsSealed}");
            Console.WriteLine($" IsPublic: {productType.IsPublic}");
            Console.WriteLine($" IsValueType: {productType.IsValueType}");

            // 1.3 Конструкторы
            Console.WriteLine("\n1.3 Конструкторы:");
            foreach (ConstructorInfo ctor in productType.GetConstructors())
            {
                var parameters = string.Join(", ",
                    ctor.GetParameters().Select(p => $"{p.ParameterType.Name}
{p.Name}"));
                Console.WriteLine($" {ctor.Name}({parameters}) [Public:
{ctor.IsPublic}]");
            }

            // 1.4 Методы (только первые 10)
            Console.WriteLine("\n1.4 Методы:");
            int methodCount = 0;
            foreach (MethodInfo method in productType.GetMethods())
            {
```

```
                if (methodCount++ < 10)
                {
                    var params_str = string.Join(", ",
                        method.GetParameters().Select(p =>
$"{p.ParameterType.Name} {p.Name}"));
                    Console.WriteLine($" {method.ReturnType.Name}
{method.Name}({params_str})");
                }
            }
            Console.WriteLine($" ... и еще {productType.GetMethods().Length -
10} методов");

            // 1.5 Свойства
            Console.WriteLine("\n1.5 Свойства:");
            foreach (PropertyInfo prop in productType.GetProperties())
            {
                string access = $"{(prop.CanRead ? "get" :
"")}{(prop.CanWrite ? "set" : "")}";
                Console.WriteLine($" {prop.PropertyType.Name} {prop.Name} {{
{access} }}");
            }

            // 1.6 Поля (включая приватные)
            Console.WriteLine("\n1.6 Поля:");
            foreach (FieldInfo field in
productType.GetFields(BindingFlags.Public |

BindingFlags.NonPublic |

BindingFlags.Instance |

BindingFlags.Static))
            {
                string visibility = field.IsPublic ? "public" :
(field.IsPrivate ? "private" : "protected");
                string isStatic = field.IsStatic ? "static" : "";
                Console.WriteLine($" {visibility} {isStatic}
{field.FieldType.Name} {field.Name}");
            }

            Console.WriteLine("\nНажмите любую клавишу для продолжения...");
            Console.ReadKey();
        }
    }

    // ====================== Работа с приватными членами (методичка)
======================
    class Task2
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 2: РАБОТА С ПРИВАТНЫМИ ЧЛЕНАМИ
===\n");

            // Создаем экземпляр Product
            Product product = new Product(1, "Laptop", 1000.0);
            Type type = typeof(Product);

            // 2.1 Получение приватного поля _price
            Console.WriteLine("2.1 Работа с приватным полем _price:");
            FieldInfo priceField = type.GetField("_price",
                BindingFlags.NonPublic | BindingFlags.Instance);
```

```
            if (priceField != null)
            {
                double oldValue = (double)priceField.GetValue(product);
                Console.WriteLine($" Текущее значение _price: {oldValue}");

                // Изменяем значение
                priceField.SetValue(product, 1500.0);
                double newValue = (double)priceField.GetValue(product);
                Console.WriteLine($" Новое значение _price: {newValue}");
            }

            // 2.2 Вызов приватного метода ValidatePrice
            Console.WriteLine("\n2.2 Вызов приватного метода
ValidatePrice:");
            MethodInfo validateMethod = type.GetMethod("ValidatePrice",
                BindingFlags.NonPublic | BindingFlags.Instance);

            if (validateMethod != null)
            {
                // Устанавливаем отрицательную цену
                priceField.SetValue(product, -500.0);
                Console.WriteLine($" Цена до валидации:
{priceField.GetValue(product)}");

                // Вызываем приватный метод
                validateMethod.Invoke(product, null);
                Console.WriteLine($" Цена после валидации:
{priceField.GetValue(product)}");
            }

            // 2.3 Получение приватного поля _name
            Console.WriteLine("\n2.3 Работа с приватным полем _name:");
            FieldInfo nameField = type.GetField("_name",
                BindingFlags.NonPublic | BindingFlags.Instance);

            if (nameField != null)
            {
                string currentName = (string)nameField.GetValue(product);
                Console.WriteLine($" Текущее имя: {currentName}");

                nameField.SetValue(product, "Gaming Laptop");
                Console.WriteLine($" Новое имя:
{nameField.GetValue(product)}");
            }

            // 2.4 Вызов защищенного метода
            Console.WriteLine("\n2.4 Вызов защищенного метода
UpdateCategory:");
            MethodInfo updateCategory = type.GetMethod("UpdateCategory",
                BindingFlags.NonPublic | BindingFlags.Instance);

            if (updateCategory != null)
            {
                PropertyInfo categoryProp = type.GetProperty("Category");
                Console.WriteLine($" Категория до:
{categoryProp.GetValue(product)}");

                updateCategory.Invoke(product, new object[] { "Electronics"
});
                Console.WriteLine($" Категория после:
{categoryProp.GetValue(product)}");
```

```
            }

            Console.WriteLine("\nНажмите любую клавишу для продолжения...");
            Console.ReadKey();
        }
    }

    // ====================== Позднее связывание (методичка)
====================

    // Класс-калькулятор для демонстрации позднего связывания
    public class DynamicCalculator
    {
        private int _result = 0;

        public DynamicCalculator()
        {
            Console.WriteLine(" [Конструктор DynamicCalculator без
параметров]");
        }

        public DynamicCalculator(int initialValue)
        {
            _result = initialValue;
            Console.WriteLine($" [Конструктор с параметром: начальное
значение = {initialValue}]");
        }

        public int Add(int a, int b)
        {
            int res = a + b;
            Console.WriteLine($" Add({a}, {b}) = {res}");
            return res;
        }

        public double Add(double a, double b)
        {
            double res = a + b;
            Console.WriteLine($" Add({a}, {b}) = {res}");
            return res;
        }

        public int Multiply(int a, int b)
        {
            int res = a * b;
            Console.WriteLine($" Multiply({a}, {b}) = {res}");
            return res;
        }

        private void LogOperation(string operation)
        {
            Console.WriteLine($" [LOG] Выполнена операция: {operation}");
        }

        public int Result => _result;

        public void SetResult(int value)
        {
            _result = value;
            Console.WriteLine($" Установлен результат: {_result}");
        }
```

```
        public static string GetVersion()
        {
            return "DynamicCalculator v1.0";
        }
    }

    class Task3
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 3: ПОЗДНЕЕ СВЯЗЫВАНИЕ ===\n");

            // 3.1 Получение типа через имя (позднее связывание)
            Console.WriteLine("3.1 Динамическое получение типа:");
            string typeName = "LabReflection.DynamicCalculator";
            Type calcType = Type.GetType(typeName);

            if (calcType != null)
            {
                Console.WriteLine($" Тип найден: {calcType.Name}");
                Console.WriteLine($" Полное имя: {calcType.FullName}");
            }

            // 3.2 Динамическое создание экземпляра
            Console.WriteLine("\n3.2 Динамическое создание экземпляра:");

            // Через конструктор без параметров
            object calculator = Activator.CreateInstance(calcType);
            Console.WriteLine($" Экземпляр создан:
{calculator.GetType().Name}");

            // 3.3 Динамический вызов методов
            Console.WriteLine("\n3.3 Динамический вызов методов:");

            // Вызов Add(int, int)
            MethodInfo addMethod = calcType.GetMethod("Add", new[] {
typeof(int), typeof(int) });
            if (addMethod != null)
            {
                int result = (int)addMethod.Invoke(calculator, new object[] {
10, 20 });
                Console.WriteLine($" Результат Add(10,20): {result}");
            }

            // Вызов Add(double, double)
            MethodInfo addDoubleMethod = calcType.GetMethod("Add", new[] {
typeof(double), typeof(double) });
            if (addDoubleMethod != null)
            {
                double result = (double)addDoubleMethod.Invoke(calculator,
new object[] { 5.5, 3.2 });
                Console.WriteLine($" Результат Add(5.5,3.2): {result}");
            }

            // 3.4 Динамический вызов с параметрами разного типа
            Console.WriteLine("\n3.4 Вызов с параметрами разного типа:");
            MethodInfo multiplyMethod = calcType.GetMethod("Multiply");
            if (multiplyMethod != null)
            {
                int result = (int)multiplyMethod.Invoke(calculator, new
object[] { 7, 8 });
```

```
            Console.WriteLine($" Результат Multiply(7,8): {result}");
        }

        // 3.5 Работа со свойствами
        Console.WriteLine("\n3.5 Работа со свойствами через рефлексию:");
        PropertyInfo resultProp = calcType.GetProperty("Result");
        if (resultProp != null)
        {
            int currentResult = (int)resultProp.GetValue(calculator);
            Console.WriteLine($" Текущий Result: {currentResult}");
        }

        MethodInfo setResultMethod = calcType.GetMethod("SetResult");
        if (setResultMethod != null)
        {
            setResultMethod.Invoke(calculator, new object[] { 999 });
            int newResult = (int)resultProp.GetValue(calculator);
            Console.WriteLine($" Result после SetResult: {newResult}");
        }

        // 3.6 Вызов приватного метода
        Console.WriteLine("\n3.6 Вызов приватного метода:");
        MethodInfo logMethod = calcType.GetMethod("LogOperation",
            BindingFlags.NonPublic | BindingFlags.Instance);
        if (logMethod != null)
        {
            logMethod.Invoke(calculator, new object[] { "Тестовая
операция" });
            Console.WriteLine(" Приватный метод LogOperation вызван
успешно");
        }

        // 3.7 Вызов статического метода
        Console.WriteLine("\n3.7 Вызов статического метода:");
        MethodInfo versionMethod = calcType.GetMethod("GetVersion",
            BindingFlags.Public | BindingFlags.Static);
        if (versionMethod != null)
        {
            string version = (string)versionMethod.Invoke(null, null);
            Console.WriteLine($" Версия: {version}");
        }

        // 3.8 Создание экземпляра через конструктор с параметром
        Console.WriteLine("\n3.8 Создание экземпляра через конструктор с
параметром:");
        ConstructorInfo ctor = calcType.GetConstructor(new[] {
typeof(int) });
        if (ctor != null)
        {
            object calcWithParam = ctor.Invoke(new object[] { 42 });
            Console.WriteLine($" Экземпляр с параметром создан");

            PropertyInfo resProp = calcType.GetProperty("Result");
            Console.WriteLine($" Начальное значение Result:
{resProp.GetValue(calcWithParam)}");
        }

        Console.WriteLine("\nНажмите любую клавишу для продолжения...");
        Console.ReadKey();
    }
}
}
```

```
// ====================== Пример инспекции интерфейса и атрибутов (методичка)
=====================
namespace LabReflection.AttributeExample
{
    // ============ АТРИБУТЫ ============
    [AttributeUsage(AttributeTargets.Class)]
    public class PluginAttribute : Attribute
    {
        public string Name { get; }
        public PluginAttribute(string name) => Name = name;
    }

    [AttributeUsage(AttributeTargets.Method)]
    public class PluginMethodAttribute : Attribute
    {
        public string Description { get; }
        public PluginMethodAttribute(string description) => Description =
description;
    }

    // ============ ИНТЕРФЕЙС ============
    public interface IPlugin
    {
        void Execute();
    }

    // ============ ПЛАГИН С АТРИБУТАМИ ============
    [Plugin("MySimplePlugin")]
    public class MyPlugin : IPlugin
    {
        [PluginMethod("Основной метод")]
        public void Execute()
        {
            Console.WriteLine("Привет из плагина!");
        }

        [PluginMethod("Метод для сложения")]
        public int Add(int a, int b)
        {
            return a + b;
        }
    }

    // ============ ГЛАВНЫЙ КЛАСС С РЕФЛЕКСИЕЙ ============
    class Program
    {
        public static void Run()
        {
            Console.WriteLine("=== ДЕМОНСТРАЦИЯ РЕФЛЕКСИИ ===\n");

            // 1. Создаем экземпляр плагина
            IPlugin plugin = new MyPlugin();
            Type type = plugin.GetType();

            // 2. Читаем атрибут класса
            PluginAttribute classAttr =
type.GetCustomAttribute<PluginAttribute>();
            Console.WriteLine($"Имя плагина: {classAttr?.Name ?? "Не
указано"}");

            // 3. Получаем все методы
```

```
            MethodInfo[] methods = type.GetMethods(BindingFlags.Public |
BindingFlags.Instance);

            Console.WriteLine("\nМетоды плагина:");
            foreach (MethodInfo method in methods)
            {
                // Пропускаем методы Object (ToString, GetHashCode, etc)
                if (method.DeclaringType == typeof(object))
                    continue;

                // Читаем атрибут метода
                PluginMethodAttribute methodAttr =
method.GetCustomAttribute<PluginMethodAttribute>();

                Console.WriteLine($"\nМетод: {method.Name}");
                if (methodAttr != null)
                    Console.WriteLine($" Описание:
{methodAttr.Description}");

                // 4. Вызываем метод через Invoke
                try
                {
                    if (method.Name == "Execute")
                    {
                        // Метод без параметров
                        method.Invoke(plugin, null);
                    }
                    else if (method.Name == "Add")
                    {
                        // Метод с параметрами
                        object result = method.Invoke(plugin, new object[] {
5, 3 });

                        Console.WriteLine($" Результат: {result}");
                    }
                }
                catch (Exception ex)
                {
                    Console.WriteLine($" Ошибка: {ex.InnerException?.Message
?? ex.Message}");
                }
            }

            // 5. Дополнительный пример - динамический вызов по имени
            Console.WriteLine("\n=== ДИНАМИЧЕСКИЙ ВЫЗОВ ===");
            CallMethodByName(plugin, "Add", 10, 20);
            CallMethodByName(plugin, "Execute");
        }

        // Универсальный метод для вызова любого метода по имени
        static void CallMethodByName(IPlugin plugin, string methodName,
params object[] parameters)
        {
            Type type = plugin.GetType();
            MethodInfo method = type.GetMethod(methodName);

            if (method != null)
            {
                Console.WriteLine($"\nВызов {methodName}()");
                object result = method.Invoke(plugin, parameters);
                if (result != null)
                    Console.WriteLine($" Результат: {result}");
            }
```

```
            else
            {
                Console.WriteLine($"\nМетод {methodName} не найден");
            }
        }
    }
}

// ====================== Задания 1-7 ======================
namespace LabReflection.Tasks
{
    using LabReflection.AttributeExample;

    // Задание 1. Класс с приватными полями и свойствами с валидацией
    public class Employee
    {
        private string _name;
        private int _age;
        private double _salary;

        public static int Count = 0;

        public Employee()
        {
            _name = "Без имени";
            _age = 18;
            _salary = 0;
            Count++;
        }

        public Employee(string name, int age, double salary)
        {
            Name = name;
            Age = age;
            Salary = salary;
            Count++;
        }

        public string Name
        {
            get => _name;
            set
            {
                if (string.IsNullOrWhiteSpace(value))
                    throw new ArgumentException("Имя не может быть пустым");
                _name = value;
            }
        }

        public int Age
        {
            get => _age;
            set
            {
                if (value < 18 || value > 70)
                    throw new ArgumentOutOfRangeException(nameof(Age),
"Возраст должен быть от 18 до 70");
                _age = value;
            }
        }

        public double Salary
```

```
        {
            get => _salary;
            set
            {
                if (value < 0)
                    throw new ArgumentOutOfRangeException(nameof(Salary),
"Зарплата не может быть отрицательной");
                _salary = value;
            }
        }

        public void PrintInfo()
        {
            Console.WriteLine($" Employee: {_name}, {_age} лет, зарплата
{_salary}");
        }

        public double CalculateBonus(double percent)
        {
            return _salary * percent / 100;
        }

        private bool IsSenior()
        {
            return _age >= 45;
        }

        protected void RaiseSalary(double amount)
        {
            _salary += amount;
        }

        public static int GetCount()
        {
            return Count;
        }
    }

    class Task1
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 1: КЛАСС С ПРИВАТНЫМИ ПОЛЯМИ И
СВОЙСТВАМИ ===\n");

            var employee = new Employee("Иван", 30, 1500);
            employee.PrintInfo();

            Console.WriteLine("\nПроверка валидации свойств:");
            try { employee.Age = 15; }
            catch (ArgumentException ex) { Console.WriteLine($" Age = 15:
{ex.Message}"); }

            try { employee.Salary = -100; }
            catch (ArgumentException ex) { Console.WriteLine($" Salary = -
100: {ex.Message}"); }

            try { employee.Name = ""; }
            catch (ArgumentException ex) { Console.WriteLine($" Name = \"\":
{ex.Message}"); }

            Console.WriteLine("\nПриватные поля класса (через рефлексию):");
```

```
            foreach (FieldInfo field in
typeof(Employee).GetFields(BindingFlags.NonPublic | BindingFlags.Instance))
            {
                Console.WriteLine($" private {field.FieldType.Name}
{field.Name} = {field.GetValue(employee)}");
            }
        }
    }

    // Задание 2. Информация о всех методах класса
    class Task2
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 2: ИНФОРМАЦИЯ О МЕТОДАХ КЛАССА
===\n");

            Type type = typeof(Employee);

            MethodInfo[] methods = type.GetMethods(BindingFlags.Public |
                                                   BindingFlags.NonPublic |
                                                   BindingFlags.Instance |
                                                   BindingFlags.Static |

BindingFlags.DeclaredOnly);

            Console.WriteLine("Методы Employee (без get_ и set_):");
            foreach (MethodInfo method in methods)
            {
                // Фильтруем методы, сгенерированные для свойств
                if (method.IsSpecialName && (method.Name.StartsWith("get_")
|| method.Name.StartsWith("set_")))
                    continue;

                string access = method.IsPublic ? "public"
                              : method.IsPrivate ? "private"
                              : method.IsFamily ? "protected"
                              : "internal";
                string isStatic = method.IsStatic ? "static " : "";
                var parameters = string.Join(", ",
                    method.GetParameters().Select(p =>
$"{p.ParameterType.Name} {p.Name}"));

                Console.WriteLine($" {access}
{isStatic}{method.ReturnType.Name} {method.Name}({parameters})");
            }

            Console.WriteLine("\nРазница между флагами привязки:");
            PrintCount(type, "GetMethods() без флагов", type.GetMethods());
            PrintCount(type, "Public | Instance",
type.GetMethods(BindingFlags.Public | BindingFlags.Instance));
            PrintCount(type, "NonPublic | Instance",
type.GetMethods(BindingFlags.NonPublic | BindingFlags.Instance));
            PrintCount(type, "Public | Static",
type.GetMethods(BindingFlags.Public | BindingFlags.Static));
            PrintCount(type, "Public | Instance | DeclaredOnly",
                type.GetMethods(BindingFlags.Public | BindingFlags.Instance |
BindingFlags.DeclaredOnly));
        }

        static void PrintCount(Type type, string flags, MethodInfo[] methods)
        {
```

```
            Console.WriteLine($" {flags,-34}: {methods.Length,2} методов ->
{string.Join(", ", methods.Select(m => m.Name).Take(6))}{(methods.Length > 6
? ", ..." : "")}");
        }
    }

    // Задание 3. Создание экземпляра через Activator.CreateInstance
    class Task3
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 3: ACTIVATOR.CREATEINSTANCE
===\n");

            Type type = typeof(Employee);

            // Способ 1: конструктор без параметров
            object emp1 = Activator.CreateInstance(type);
            Console.WriteLine("1. Создан объект через конструктор без
параметров:");
            PrintProperties(emp1);

            // Установка свойств через рефлексию
            type.GetProperty("Name").SetValue(emp1, "Анна");
            type.GetProperty("Age").SetValue(emp1, 25);
            type.GetProperty("Salary").SetValue(emp1, 2000.0);
            Console.WriteLine("\n   После установки свойств через
SetValue:");
            PrintProperties(emp1);

            // Способ 2: конструктор с параметрами
            object emp2 = Activator.CreateInstance(type, new object[] {
"Петр", 40, 3500.0 });
            Console.WriteLine("\n2. Создан объект через конструктор с
параметрами:");
            PrintProperties(emp2);

            int count = (int)type.GetMethod("GetCount").Invoke(null, null);
            Console.WriteLine($"\nВсего создано объектов Employee: {count}");
        }

        static void PrintProperties(object obj)
        {
            foreach (PropertyInfo prop in obj.GetType().GetProperties())
            {
                Console.WriteLine($"   {prop.Name} = {prop.GetValue(obj)}");
            }
        }
    }

    // Задание 4. "Черный ящик" с приватным полем и приватными методами
    public class SecretBox
    {
        private int _secret = 42;

        public int GetSecret()
        {
            return _secret;
        }

        private int Calculate(int a, int b)
        {
```

```
            return a * b + _secret;
        }

        private int Square()
        {
            return _secret * _secret;
        }

        private string Describe(string prefix)
        {
            return $"{prefix}: секрет = {_secret}";
        }
    }

    // Задание 5. Доступ к приватному полю и изменение его значения
    class Task5
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЯ 4 И 5: ДОСТУП К ПРИВАТНОМУ ПОЛЮ
===\n");

            var box = new SecretBox();
            Type type = typeof(SecretBox);
            Console.WriteLine($"Значение через публичный метод GetSecret():
{box.GetSecret()}");

            try
            {
                FieldInfo secretField = type.GetField("_secret",
BindingFlags.NonPublic | BindingFlags.Instance);
                if (secretField == null)
                    throw new MissingFieldException(type.Name, "_secret");

                Console.WriteLine($"Значение через FieldInfo.GetValue:
{secretField.GetValue(box)}");

                secretField.SetValue(box, 100);
                Console.WriteLine($"После SetValue(100) GetSecret() вернул:
{box.GetSecret()}");
                Console.WriteLine("Инкапсуляция нарушена: приватное поле
изменено извне.");
            }
            catch (FieldAccessException ex)
            {
                Console.WriteLine($"Недостаточно прав: {ex.Message}");
            }

            Console.WriteLine("\nОбработка исключений:");
            try
            {
                FieldInfo wrongField = type.GetField("_unknown",
BindingFlags.NonPublic | BindingFlags.Instance);
                if (wrongField == null)
                    throw new MissingFieldException(type.Name, "_unknown");
            }
            catch (MissingFieldException ex)
            {
                Console.WriteLine($" Поле не найдено: {ex.Message}");
            }

            try
```

```
            {
                FieldInfo secretField = type.GetField("_secret",
BindingFlags.NonPublic | BindingFlags.Instance);
                secretField.SetValue(box, "текст");
            }
            catch (ArgumentException ex)
            {
                Console.WriteLine($" Неверный тип значения: {ex.Message}");
            }
        }
    }

    // Задание 6. Вызов приватных методов через рефлексию
    class Task6
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 6: ВЫЗОВ ПРИВАТНОГО МЕТОДА
===\n");

            var box = new SecretBox();
            Type type = typeof(SecretBox);
            BindingFlags flags = BindingFlags.NonPublic |
BindingFlags.Instance;

            Console.WriteLine("Способ 1: MethodInfo.Invoke");
            MethodInfo square = type.GetMethod("Square", flags);
            Console.WriteLine($" Square() = {square.Invoke(box, null)}");

            MethodInfo describe = type.GetMethod("Describe", flags);
            Console.WriteLine($" Describe(\"Ящик\") = {describe.Invoke(box,
new object[] { "Ящик" })}");

            MethodInfo calculate = type.GetMethod("Calculate", flags);
            Console.WriteLine($" Calculate(3, 5) = {calculate.Invoke(box, new
object[] { 3, 5 })}");

            Console.WriteLine("\nСпособ 2: создание делегата");
            var calcDelegate = (Func<int, int,
int>)calculate.CreateDelegate(typeof(Func<int, int, int>), box);
            Console.WriteLine($" calcDelegate(3, 5) = {calcDelegate(3, 5)}");

            var squareDelegate = square.CreateDelegate<Func<int>>(box);
            Console.WriteLine($" squareDelegate() = {squareDelegate()}");

            Console.WriteLine("\nПриватные методы SecretBox:");
            foreach (MethodInfo m in type.GetMethods(flags |
BindingFlags.DeclaredOnly))
            {
                Console.WriteLine($" private {m.ReturnType.Name}
{m.Name}({m.GetParameters().Length} параметр.)");
            }
            Console.WriteLine("Приватные методы доступны через рефлексию
вопреки модификатору private.");
        }
    }

    // Задание 7. Плагины для динамической загрузки
    [Plugin("Greeting")]
    public class GreetingPlugin : IPlugin
    {
        public void Execute()
```

```
        {
            Console.WriteLine("  Привет от GreetingPlugin!");
        }
    }

    [Plugin("Reverse")]
    public class ReversePlugin : IPlugin
    {
        public void Execute()
        {
            string text = "Рефлексия";
            Console.WriteLine($"  {text} -> {new
string(text.Reverse().ToArray())}");
        }
    }

    // Реализует IPlugin, но не помечен атрибутом [Plugin] - будет
отфильтрован
    public class NoAttributePlugin : IPlugin
    {
        public void Execute()
        {
            Console.WriteLine("  Этот плагин не должен загрузиться");
        }
    }

    class Task7
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 7: ДИНАМИЧЕСКАЯ ЗАГРУЗКА ПЛАГИНОВ
===\n");

            Assembly assembly = Assembly.GetExecutingAssembly();

            // Поиск всех классов, реализующих IPlugin и помеченных [Plugin]
            var pluginTypes = assembly.GetTypes()
                .Where(t => typeof(IPlugin).IsAssignableFrom(t) &&
!t.IsInterface && !t.IsAbstract)
                .Where(t => t.GetCustomAttribute<PluginAttribute>() != null)
                .ToList();

            Console.WriteLine("Найдены плагины в сборке:");
            foreach (Type t in pluginTypes)
            {
                Console.WriteLine($" {t.FullName}
[Plugin(\"{t.GetCustomAttribute<PluginAttribute>().Name}\")]");
            }

            // Загрузка по именам классов из конфигурационного файла
            string configPath = Path.Combine(AppContext.BaseDirectory,
"plugins.config");
            string[] names = File.ReadAllLines(configPath)
                .Select(line => line.Trim())
                .Where(line => line.Length > 0 && !line.StartsWith("#"))
                .ToArray();

            Console.WriteLine($"\nКлассы из plugins.config: {names.Length}");
            foreach (string name in names)
            {
                Type type = pluginTypes.FirstOrDefault(t => t.FullName ==
name);
```

```
                if (type == null)
                {
                    Console.WriteLine($" {name}: не найден среди плагинов");
                    continue;
                }

                IPlugin plugin = (IPlugin)Activator.CreateInstance(type);
                Console.WriteLine($" Загружен
{type.GetCustomAttribute<PluginAttribute>().Name}:");
                plugin.Execute();
            }

            Console.WriteLine("\nДля нового плагина достаточно добавить класс
с [Plugin] и строку в plugins.config.");
        }
    }
}

// ==================== Дополнительно: система плагинов (методичка)
====================
namespace LabReflection.PluginSystem
{
    // Общий интерфейс для всех плагинов
    public interface IPlugin
    {
        string Name { get; }
        string Version { get; }
        void Execute();
    }

    // Дополнительный интерфейс для плагинов с параметрами
    public interface IParameterizedPlugin : IPlugin
    {
        void ExecuteWithParams(Dictionary<string, object> parameters);
    }

    // Плагин 1: Простой калькулятор
    public class CalculatorPlugin : IPlugin
    {
        public string Name => "Calculator";
        public string Version => "1.0.0";

        public void Execute()
        {
            Console.WriteLine("Calculator plugin loaded");
        }

        public double Add(double a, double b) => a + b;
        public double Subtract(double a, double b) => a - b;
    }

    // Плагин 2: Логгер с параметрами
    public class LoggerPlugin : IParameterizedPlugin
    {
        public string Name => "Logger";
        public string Version => "1.0.0";

        public void Execute()
        {
            Console.WriteLine("Logger plugin executed");
        }
```

```
        public void ExecuteWithParams(Dictionary<string, object> parameters)
        {
            if (parameters.TryGetValue("message", out var msg))
            {
                Console.WriteLine($"[LOG] {msg}");
            }
        }
    }

    public class PluginManager
    {
        private readonly List<IPlugin> _plugins = new List<IPlugin>();
        private readonly string _pluginsPath;

        public PluginManager(string pluginsPath = null)
        {
            _pluginsPath = pluginsPath ??
Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "Plugins");
            Directory.CreateDirectory(_pluginsPath);
        }

        // Загрузка всех плагинов из директории
        public void LoadPlugins()
        {
            var dllFiles = Directory.GetFiles(_pluginsPath, "*.dll");

            foreach (var dllPath in dllFiles)
            {
                try
                {
                    LoadPluginFromAssembly(dllPath);
                }
                catch (Exception ex)
                {
                    Console.WriteLine($"Error loading plugin {dllPath}:
{ex.Message}");
                }
            }
        }

        // Загрузка встроенных плагинов из текущей сборки
        public void LoadBuiltInPlugins()
        {
            Console.WriteLine("Loading built-in plugins...");
            LoadPluginsFromTypes(Assembly.GetExecutingAssembly().GetTypes());
        }

        // Загрузка плагина из конкретной сборки
        private void LoadPluginFromAssembly(string assemblyPath)
        {
            // Загрузка сборки
            Assembly assembly = Assembly.LoadFrom(assemblyPath);

            // Получение всех типов в сборке
            Type[] types = assembly.GetTypes();
            LoadPluginsFromTypes(types);
        }

        private void LoadPluginsFromTypes(Type[] types)
        {
            foreach (var type in types)
            {
```

```
                // Проверка, реализует ли тип интерфейс IPlugin
                if (typeof(IPlugin).IsAssignableFrom(type) &&
!type.IsInterface && !type.IsAbstract)
                {
                    // Создание экземпляра плагина
                    IPlugin plugin = (IPlugin)Activator.CreateInstance(type);
                    _plugins.Add(plugin);
                    Console.WriteLine($"Loaded plugin: {plugin.Name}
v{plugin.Version}");
                }
            }
        }

        // Получение всех загруженных плагинов
        public IEnumerable<IPlugin> GetPlugins() => _plugins;

        // Получение плагина по имени
        public IPlugin GetPlugin(string name)
        {
            return _plugins.FirstOrDefault(p => p.Name.Equals(name,
StringComparison.OrdinalIgnoreCase));
        }

        // Выполнение плагина с параметрами через рефлексию
        public void ExecutePluginMethod(IPlugin plugin, string methodName,
object[] parameters)
        {
            Type pluginType = plugin.GetType();

            // Поиск метода
            MethodInfo method = pluginType.GetMethod(methodName);

            if (method != null)
            {
                // Вызов метода
                var result = method.Invoke(plugin, parameters);
                Console.WriteLine($"Method {methodName} executed. Result:
{result ?? "null"}");
            }
            else
            {
                Console.WriteLine($"Method {methodName} not found in plugin
{plugin.Name}");
            }
        }

        // Вызов метода с параметрами через рефлексию с проверкой типов
        public T ExecutePluginMethod<T>(IPlugin plugin, string methodName,
object[] parameters)
        {
            Type pluginType = plugin.GetType();
            MethodInfo method = pluginType.GetMethod(methodName);

            if (method != null && method.ReturnType == typeof(T))
            {
                return (T)method.Invoke(plugin, parameters);
            }

            return default;
        }
    }
```

```
    class Program
    {
        public static void Run()
        {
            Console.WriteLine("=== Plugin System Demo ===\n");

            var manager = new PluginManager();
            manager.LoadPlugins();
            manager.LoadBuiltInPlugins();

            Console.WriteLine("\n=== Available Plugins ===");
            foreach (var plugin in manager.GetPlugins())
            {
                Console.WriteLine($"- {plugin.Name} (v{plugin.Version})");
                if (plugin is IParameterizedPlugin)
                    Console.WriteLine("  Supports parameters");
            }

            Console.WriteLine("\n=== Plugin Demonstrations ===");

            Console.WriteLine("\nTesting Calculator plugin:");
            IPlugin calculator = manager.GetPlugin("Calculator");
            calculator.Execute();
            Console.WriteLine("  Using reflection to call methods:");
            double sum = manager.ExecutePluginMethod<double>(calculator,
"Add", new object[] { 15.5, 24.7 });
            Console.WriteLine($"  15.5 + 24.7 = {sum}");
            double diff = manager.ExecutePluginMethod<double>(calculator,
"Subtract", new object[] { 50.0, 12.3 });
            Console.WriteLine($"  50.0 - 12.3 = {diff}");

            Console.WriteLine("\nTesting Logger plugin:");
            IPlugin logger = manager.GetPlugin("Logger");
            logger.Execute();
            if (logger is IParameterizedPlugin parameterized)
            {
                Console.WriteLine("  Using parameterized execution:");
                parameterized.ExecuteWithParams(new Dictionary<string,
object> { ["message"] = "Hello from Plugin System!" });
            }
            Console.WriteLine("  Using reflection with parameters:");
            manager.ExecutePluginMethod(logger, "ExecuteWithParams",
                new object[] { new Dictionary<string, object> { ["message"] =
"Reflection call!" } });

            Console.WriteLine("\n=== Error Handling Demo ===");
            Console.WriteLine("  Trying to call non-existent method:");
            manager.ExecutePluginMethod(calculator, "NonExistentMethod",
null);
            Console.WriteLine("  Trying to get non-existent plugin:");
            Console.WriteLine($"  GetPlugin(\"Unknown\") =
{manager.GetPlugin("Unknown")?.Name ?? "null"}");
        }
    }
}
```

Файл *plugins.config*:

```
# Список плагинов для задания 7: одно имя класса в строке.
# Чтобы подключить новый плагин, достаточно добавить сюда имя его класса.
LabReflection.AttributeExample.MyPlugin
LabReflection.Tasks.GreetingPlugin
LabReflection.Tasks.ReversePlugin
```